

Teste de Software Manual e JUnit

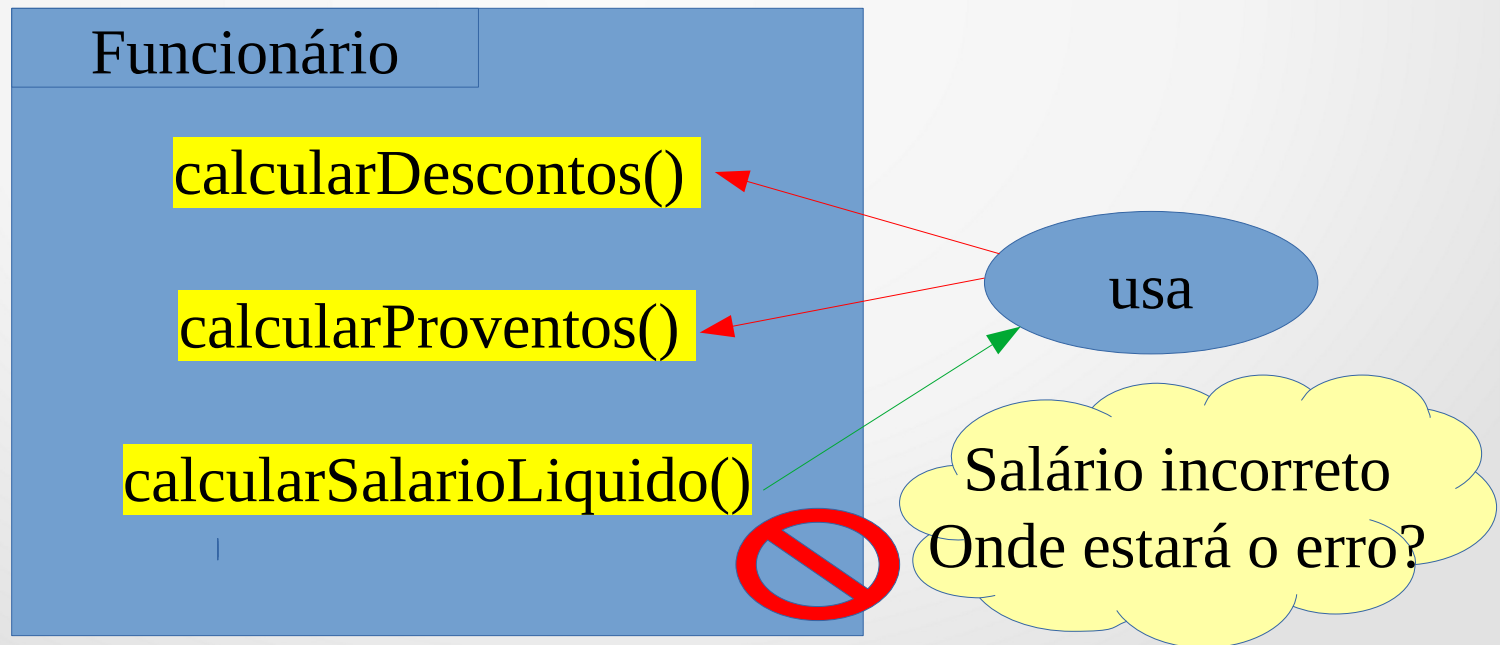
Prof. Dr. Bruno Queiroz Pinto

Testes de unidade

- Testes de unidade garantem que cada método(função) testado está produzindo o resultado esperado. Essa garantia dá uma segurança maior ao programador, que poderá mudar a implementação sem medo.

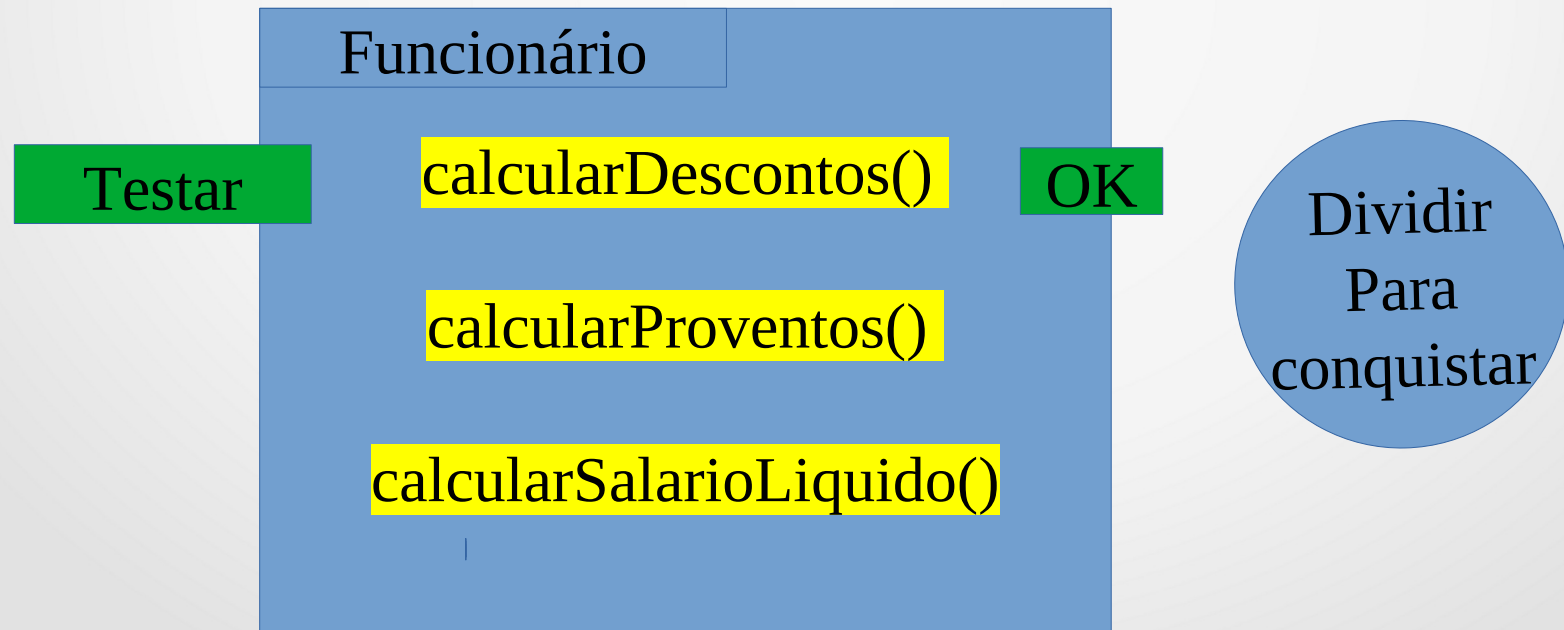
Conceito de testes unitários

➤ Teste capaz de analisar uma unidade de trabalho, que a IEEE define como: “Atividade capaz de testar unidades de hardware ou software ou grupo de unidades relacionadas”.



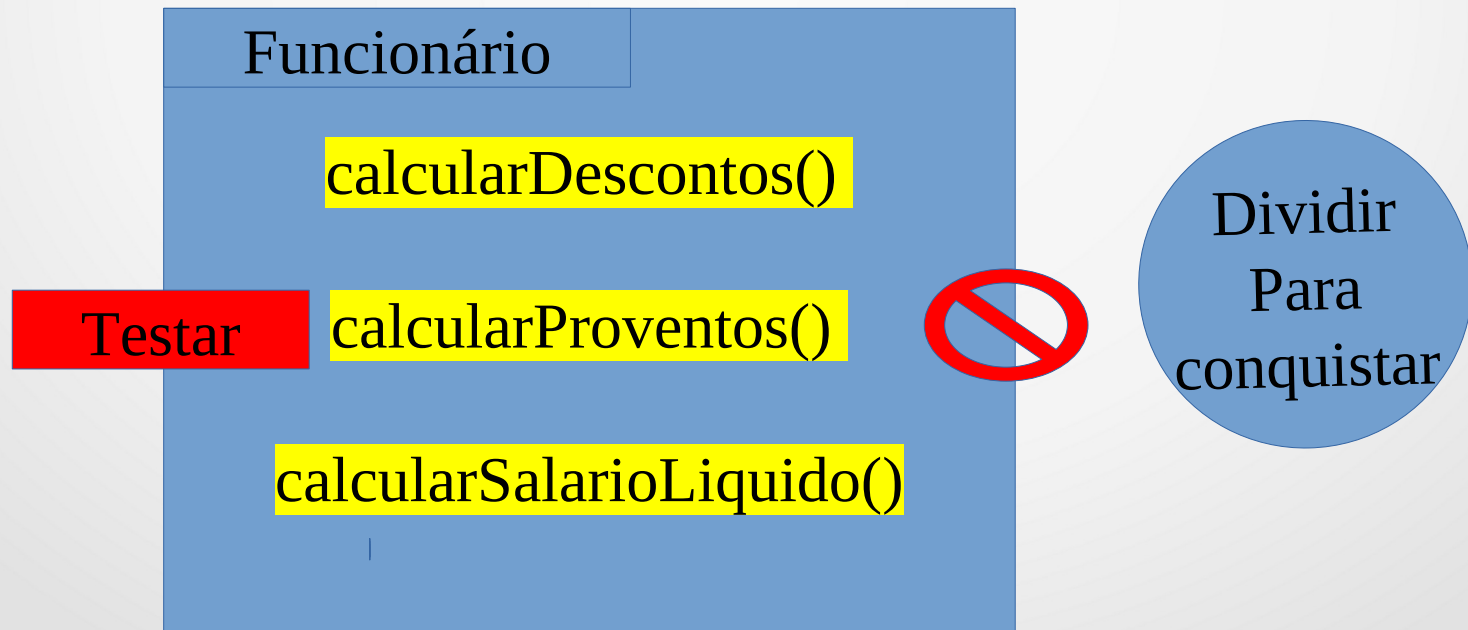
Conceito de testes unitários

➤ Teste capaz de analisar uma unidade de trabalho, que a IEEE define como: “Atividade capaz de testar unidades de hardware ou software ou grupo de unidades relacionadas”.



Conceito de testes unitários

➤ Teste capaz de analisar uma unidade de trabalho, que a IEEE define como: “Atividade capaz de testar unidades de hardware ou software ou grupo de unidades relacionadas”.



Conceito de testes unitários

➤ Analogia: Se você testou as tomadas individualmente, encontrará problemas mais facilmente.

Unidade



Unidade

Testes não automatizados (sem JUnit)

- Estudo de caso:
 - um simples aplicativo de calculadora que pode somar, subtrair, multiplicar e dividir dois números inteiros.
 - Esta aplicação terá a organização de pastas conforme mostra a Figura abaixo.

```

src
├── main
│   ├── java/org/iftm/testes/calculadorapreparacao
│   │   ├── calculadora
│   │   │   ├── Calculadora.java
│   │   │   ├── TesteManual.java
│   │   │   └── CalculadorapreparacaoApplication.java
│   │   └── resources
│   └── test/java/org/iftm/testes/calculadorapreparacao
│       ├── calculadora
│       │   ├── CalculadoraTest.java
│       │   └── CalculadorapreparacaoApplicationTests.java

```

Código da classe Calculadora

```
public class Calculadora{  
    public int somar(int num1, int num2) {  
        return num1 + num2;  
    }  
  
    public int subtrair(int num1, int num2) {  
        return num1 + num2;  
    }  
  
    public int multiplicar(int num1, int num2) {  
        return num1 + num2;  
    }  
  
    public int dividir(int num1, int num2) {  
        if (num2 == 0){  
            return -1;  
        }  
        return num1 / num2;  
    }  
}
```

Códigos com erro



Estrutura de Testes de Unidade

Estrutura de Teste

- A estrutura de Teste:

- monta o cenário,
- executa a ação
- e valida a saída.

Arrange

Act

Assert

Teste de Unidade



Tabela de Casos de Teste

Método Testado	Cenário de Teste	Entradas (num1, num2)	Resultado Esperado
somar	Dois números positivos	(2, 3)	5
somar	Número positivo e negativo	(5, -3)	2
subtrair	Subtração simples	(5, 3)	2
subtrair	Subtração com número negativo	(3, -2)	5
multiplicar	Multiplicação básica	(2, 3)	6
multiplicar	Multiplicação por zero	(5, 0)	0
dividir	Divisão inteira exata	(6, 3)	2
dividir	Divisão com resto	(5, 2)	2
dividir	Divisão por zero	(5, 0)	-1

Teste Manual

```
public class TesteManual {  
    public static void main(String args[]) {  
        /*  
        * Testa a operação de soma com dois números positivos.  
        *  
        * Cenário:Soma de valores positivos  
        * Entrada:num1 = 2, num2 = 3  
        * Saída esperada:5  
        */  
        // Arrange  
        Calculadora calc = new Calculadora();  
        int numero1 = 2;  
        int numero2 = 3;  
        int resultadoEsperado = 5;  
        // Act  
        int resultadoObtido = calc.somar(numero1, numero2);  
        // Assert  
        System.out.println(resultadoObtido);  
    }  
}
```

0.0

Instanciar um
objeto



0.0

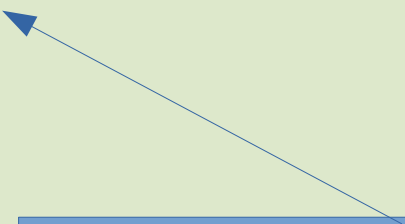
Chamar
métodos



Testes **Automatizados** em Java

Estrutura de Teste

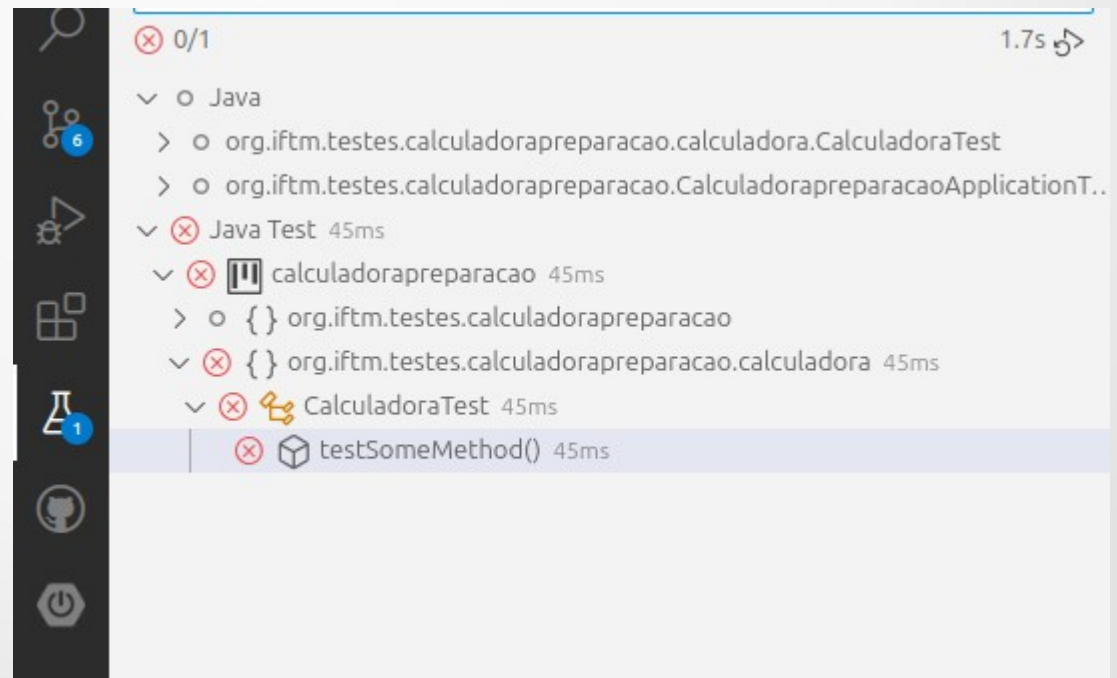
- A estrutura de Teste quase não modifica:
 - monta o cenário,
 - executa a ação
 - **e valida a saída.**



Modifica aqui : automatizado

Testes Automatizados com JUnit

- JUnit, o mais popular framework de testes de unidade para Java.
- O framework apresenta quais casos de testes falharam e qual foi a saída incorreta produzida pelo método.



Testes Automatizados com JUnit

Nosso código anterior está muito perto de ser entendido pelo JUnit. Precisamos fazer apenas algumas mudanças:

1. Classe de Teste deve ter o nome da classe + palavra **Test**.
2. Deve ser anotado com `@Test` .
3. Deve ter um nome que indica o que será testado.

ex.:

`deveSomaDoisInteirosPositivos`

ou

`testarSomaDoisInteirosPositivos`

Testes Automatizados com JUnit

Automático

```
public class CalculadoraTest {  
    @Test  
    public void testaSomaDoisInteirosPositivos(){
```

Cenário
Teste

```
        Calculadora calculadora = new Calculadora();  
        int entradaNumero1 = 10;  
        int entradaNumero2 = 20;  
        int resultadoEsperado = 30;
```

Ação/Execução
Teste

```
        int resultadoObtido = calculadora.somar(entradaNumero1, entradaNumero2);
```

Avaliação

```
        // comparando a saída com o esperado
```

```
        Assertions.assertEquals(resultadoEsperado, resultadoObtido);
```

Valor esperado

Valor obtido

```
    }  
}
```

JUnit 5

Testes Automatizados com JUnit

Repare algumas mudanças no código:

- criar um package próprio para armazenar os testes.
- nome da classe: `CalculadoraTest` .
 - Convenção : `NomeDaClasseSobTesteTest`.
- nome dos métodos de teste : `testaSomaDoisInteirosPositivos`.
 - nomes relacionados ao caso de teste. Conseguimos descobrir mais facilmente o que está errado em nosso sistema.
- Validar saída .
 - Utilizar instruções `Asserts.assertEquals()` verifica se a saída esperada é igual ao resultado gerado. (manter essa ordem)

Uma forma de documentar

Caso o código detalhe claramente não é tão importante

```
/**  
 * Testa a operação de soma com dois números positivos.  
 *  
 * <p><b>Cenário:</b> Soma de valores positivos</p>  
 * <p><b>Entrada:</b> num1 = 2, num2 = 3</p>  
 * <p><b>Saída esperada:</b> 5</p>  
 */  
@Test  
public void testaSomaDoisInteirosPositivos(){
```

Geração automática de documentação: Via ferramentas como Javadoc

Podemos utilizar o @DisplayName

Nomes claros de métodos já documentam, entretanto algumas pessoas podem achar métodos muito longos difíceis de ler.

A anotação @DisplayName("Deve somar dois inteiros positivos") do JUnit 5.

Em algumas IDEs, permite gerar relatórios de teste muito mais elegantes. No Vscode não.

Facilita a leitura.

```
@Test  
@DisplayName("Deve somar dois inteiros positivos")  
public void testaSomaDoisInteirosPositivos(){
```

Testes Automatizados com JUnit



Boas Práticas para Documentação de Testes

- 1) Seja conciso, mas completo: Documente o essencial sem ser redundante
- 2) Mantenha atualizado: Atualize a documentação quando o teste mudar
- 3) Destaque cenários especiais: Divisão por zero, valores limite, etc.
- 4) Documente comportamentos esperados: Principalmente quando não óbvios



Benefícios desta Abordagem

1. Melhor manutenibilidade: Outros desenvolvedores entendem a intenção do teste
2. Facilita revisões de código: Contexto claro para cada caso de teste
3. Serve como especificação: Documenta o comportamento esperado do sistema

Implemente os testes

Testes Automatizados com JUnit

Análise dos Resultados

Testes que passarão:

- Todos os testes do método somar
- Testes do método dividir

Testes que falharão:

- Testes de subtrair (implementado incorretamente como soma)
- Testes de multiplicar (implementado incorretamente como soma)

Testes Automatizados com JUnit

Exercícios Práticos

- 1) Implemente corretamente os métodos com erros:
- 2) Reexecute o teste (passa?)
- 3) Implemente e teste um novo método `ePar(int numero)` que:
 - Retorne `true` se o número for par
 - Retorne `false` se o número for ímpar

Testes Automatizados com JUnit



Dicas para Escrever Bons Testes

- Nomes claros: Use `testar[Método][Cenário]` como convenção
- Organização: Agrupe testes relacionados na mesma classe
- Manutenção: Atualize os testes quando o código for modificado

Revisão

- 1. Por que os Testes de Unidade ficam na base e existem em maior quantidade do que os testes de Interface (E2E)?**
- 2. Quais são as três etapas do padrão AAA utilizado para estruturar nossos testes e o que cada uma faz?**
- 3. Por que um método de teste nunca deve depender do resultado de outro método de teste?**
- 4. No código Java, como o framework JUnit sabe que um determinado método não é um método comum do sistema, mas sim um cenário de teste que deve ser executado?**
- 5. Vimos que escrever testes dá trabalho. Qual é o maior benefício que o desenvolvedor ganha ao ter uma boa suíte de testes unitários quando precisa alterar ou melhorar um código antigo (refatoração)?**